

Section 83 — Implicit Treaps And Dynamic Forests

The represented order is not the storage order

This lesson develops A–C. Problem D is explained only in the editorial.

Problem A: implicit treap sequence surgery

An implicit treap is a randomized BST whose key is the in-order position, recovered from left-subtree sizes. `split(root,k)` returns the first `k` elements and the remainder; `merge(a,b)` requires every element of `a` to precede every element of `b`. Cut/paste becomes a constant number of splits and merges. Random priorities give expected logarithmic height.

Problem B: lazy sequence aggregates

Store subtree size and sum. Reversal swaps children and toggles a lazy flag; range addition changes the node value, subtree sum, and pending addition. Push tags before using children and pull aggregates after structural changes. The state boundary is exact: every delayed operation affects a whole represented interval.

Problem C: link-cut trees

A link-cut tree stores each preferred path as an auxiliary splay tree. `access(v)` repeatedly cuts and replaces right preferred children until the root-to-`v` path is exposed. `makeroot(v)` exposes that path and reverses it, changing the represented-tree root. Then `makeroot(u)`; `access(v)` makes one auxiliary tree represent exactly the `u-v` path.

The subtle distinction is between an auxiliary-tree root and a represented-tree root. Rotations change only the former. `link` and `cut` change represented edges.

Split and merge trace

For sequence `ABCDE`, `split(root,2)` returns `AB` and `CDE`. To move `BC` to the end, split into `A | BC | DE`, then merge `A`, `DE`, and `BC` in that order. No stored key is updated: subtree sizes reconstruct positions after every rotation and merge.

The precondition of `merge(left,right)` is semantic—every represented element of `left` precedes every element of `right`. Random priority chooses the storage root, but may never reorder the represented sequence.

Lazy-tag chronology

If a range is first multiplied by a and then increased by b , its pending affine tag is $x \rightarrow a*x+b$. A later tag $x \rightarrow c*x+d$ composes outside it: $x \rightarrow c*(a*x+b)+d$. Reversal is structural: swap children and forward/reverse aggregates, then toggle the flag. Push reversal before relying on which child is left or right.

Link-cut exposure trace

After `makeroot(u); access(v)`, the auxiliary splay rooted at v contains the represented path $u..v$ in order. A path aggregate is therefore the splay aggregate at v . This statement is true only after both operations; reading a random auxiliary subtree confuses preferred-path storage with represented-tree topology.

Before `link(u,v)`, verify they are in different represented trees. To cut an edge, expose $u..v$ and verify that u is the direct left child boundary of v with no intervening vertex. Rotations alone never add or remove a represented edge.

Implementation and stress testing

Treap code must push before split/merge recursion and pull afterward. Use a deterministic seeded RNG for reproducible failures. In Python, array-based nodes reduce memory overhead; iterative alternatives are harder because split and merge naturally follow tree recursion.

Link-cut correctness depends on `is_aux_root`: a node is an auxiliary root when its parent does not point to it as a splay child. Push all ancestors before splaying. Tiny forest oracles can rebuild paths with BFS after every link or cut and compare aggregates literally.

Exercises

A–C develop sequence surgery, lazy tags, and exposed forest paths. D is the reconstruction-tree transfer problem.